

Insertion Sort Incremental Algorithm

2026-03-15

1

Insertion Sort
incremental algorithm() .
Insertion Sort incremental algorithm ,
.

.

$A[1..j-1]$.
 $A[j]$.
 $A[1..j]$. , .

1
2
3
...
n

incremental algorithm .

incremental algorithm .

i Incremental Algorithm
 , .

Insertion Sort .
 ,
.

1: Insertion Sort

key			
	-	[5]	[5, 2, 4, 6, 1, 3]
1	2	[5]	[2, 5, 4, 6, 1, 3]
2	4	[2, 5]	[2, 4, 5, 6, 1, 3]
3	6	[2, 4, 5]	[2, 4, 5, 6, 1, 3]
4	1	[2, 4, 5, 6]	[1, 2, 4, 5, 6, 3]
5	3	[1, 2, 4, 5, 6]	[1, 2, 3, 4, 5, 6]

2 Loop Invariant

Insertion Sort correctness **loop invariant** .

! Loop Invariant

for , A[1..j-1] .

invariant incremental algorithm . , , .

2.1 Initialization

j = 2 .

A[1..j-1] = A[1] , .

loop invariant .

2.2 Maintenance

A[1..j-1] .

key = A[j] . key . key .

A[1..j] .

, .

2.3 Termination

j = A.length + 1 .

loop invariant A[1..A.length] , .

Insertion Sort correctness .

3 Incremental Algorithm

Insertion Sort incremental algorithm ?



Insertion Sort , .

- $A[1]$.
- $A[2]$ $A[1..2]$.
- $A[3]$ $A[1..3]$.
- ...
- $A[n]$ $A[1..n]$.

, , .
incremental algorithm .

4

Insertion Sort .

4.1 Best Case

$A[i] > \text{key}$, while .
 ,

$O(n)$

4.2 Worst Case

[6, 5, 4, 3, 2, 1]

$$1 + 2 + 3 + \dots + (n - 1)$$

$$O(n^2)$$

4.3

Insertion Sort

- 1 ,
- 2 ,
- 3 ,
- ...
- (n-1) (n-1)

5

Insertion Sort incremental

5.1 Incremental vs Divide-and-Conquer

5.1.1 Incremental

-
-
- : Insertion Sort

5.1.2 Divide-and-Conquer

-
-
-
- : Merge Sort

, Insertion Sort

Insertion Sort incremental algorithm

6 Pseudo code

Insertion Sort pseudo code .

```
INSERTION-SORT( $A$ )
1  for  $j = 2$  to  $A.length$ 
2    key =  $A[j]$ 
3    // Insert  $A[j]$  into the sorted sequence  $A[1 \dots j - 1]$ 
4     $i = j - 1$ 
5    while  $i > 0$  and  $A[i] > \text{key}$ 
6       $A[i + 1] = A[i]$ 
7       $i = i - 1$ 
8     $A[i + 1] = \text{key}$ 
```

7 Conclusion

Insertion Sort incremental algorithm ,

Loop invariant correctness ,
Insertion Sort , incremental algorithm .

8 Codes

```
#include <stdio.h>
#include <vector>

void _sort_func(std::vector<int>& a) {
    int n = a.size();
    for (int j = 1; j < n; ++j) {
        int key = a[j];
        int i = j - 1;
        while (i >= 0 && a[i] > key) {
            a[i + 1] = a[i];
            --i;
        }
        a[i + 1] = key;
    }
}
```